Day 166 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 24, Day 6 reference

Watch: Fairness, part 1 - Moritz Hardt - MLSS 2020, Tuebingen (virtual mlss2020) | FAT* 2018 Translation Tutorial: 21 Definitions of Fairness and Their Politics (ACM FAccT)

`day166_fairness.py`, segment by segment, in the order `main()` uses things. The parts (group_rates, gap, threshold_for_rate, threshold_for_tpr, decide, fpr_from_identity, feature_of_column) are copied unchanged into `day166_exercise.py`; the Adult data and logistic regression (Day 161), boosting (Day 109), ROC AUC (Day 117), the bootstrap and Wilson interval (Day 163) and the ECE (Day 165) are imported from their exercise files. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: y = 0/1 outcomes · d = 0/1 decisions · groups, sv = each row's sex · p = predicted probabilities · thr = group → threshold · A, Bh = the two halves of the validation rows

Segment 0 — Setup

Imports and constants

CODE

```
sys.path.insert(0, os.path.join(ROOT, "week-16-classical-ml-trees"))
sys.path.insert(0, os.path.join(ROOT, "week-17-classical-ml-kernels-ensembles"))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day109_exercise as d109
    import day117_exercise as d117
    import day161_exercise as d161
    import day163_exercise as d163
    import day165_exercise as d165
GROUPS = ("Female", "Male")
METRICS = ("sel", "tpr", "fpr", "ppv")
B = 1000
SPLITS = 50
```

- `import day163_exercise as d163` — Day 163's bootstrap and Wilson interval, unchanged.
- `GROUPS = ("Female", "Male")` — a tuple: the order every loop and figure uses.
- `METRICS = ("sel", "tpr", "fpr", "ppv")` — the dictionary keys of the four rates.

Segment 1 — The parts

group_rates and gap: a dictionary of dictionaries

MATH

$$TPR = \text{mean of } d \text{ where } y = 1 \quad FPR = \text{mean of } d \text{ where } y = 0 \quad precision = \text{mean of } y \text{ where } d = 1$$

CODE

```
def group_rates(y, d, groups):
    out = {}
    for g in np.unique(groups):
        yy, dd = y[groups == g], d[groups == g]
        out[g] = {"n": len(yy), "base": yy.mean(),
                  "sel": dd.mean(),
                  "tpr": dd[yy == 1].mean() if (yy == 1).any() else np.nan,
                  "fpr": dd[yy == 0].mean() if (yy == 0).any() else np.nan,
                  "ppv": yy[dd].mean() if dd.any() else np.nan}
    return out
def gap(y, d, groups, metric, a="Male", b="Female"):
    r = group_rates(y, d, groups)
    return r[a][metric] - r[b][metric]
```

- `for g in np.unique(groups):` — the distinct group names, sorted.
- `y[groups == g]` — boolean indexing: the outcomes of this group's rows only.
- `dd[yy == 1].mean() if (yy == 1).any() else np.nan` — a conditional expression: the TPR, or NaN when the group has no positives (the mean of an empty array would warn).
- `yy[dd].mean()` — d is boolean, so yy[dd] keeps the outcomes of the selected rows: the precision.
- `r[a][metric]` — two lookups: the group's dictionary, then the rate inside it.
- `a="Male", b="Female"` — default arguments: the gap is men minus women unless told otherwise.

threshold_for_rate, threshold_for_tpr and decide

CODE

```
def threshold_for_rate(p, rate):
    return np.quantile(p, 1 - rate)
def threshold_for_tpr(p, y, tpr):
    return np.quantile(p[y == 1], 1 - tpr)
def decide(p, groups, thr):
    d = np.zeros(len(p), bool)
    for g, t in thr.items():
        d[groups == g] = p[groups == g] > t
    return d
```

- `np.quantile(p, 1 - rate)` — the score with a share 1 – rate below it, so a share rate above it.
- `np.quantile(p[y == 1], 1 - tpr)` — the same, using only the truly positive rows' scores.
- `np.zeros(len(p), bool)` — an array of False, one per row.
- `for g, t in thr.items():` — each key-value pair of the dictionary: a group and its threshold.
- `d[groups == g] = p[groups == g] > t` — assigning through a boolean mask writes only this group's rows.

fpr_from_identity and feature_of_column

MATH

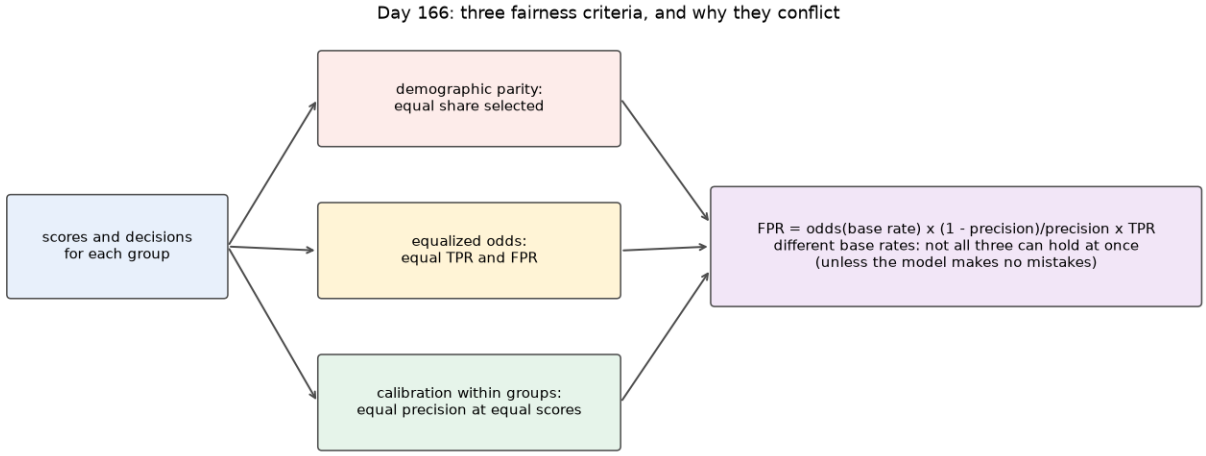
$$FPR = base/(1 - base) \cdot (1 - precision)/precision \cdot TPR$$

CODE

```
def fpr_from_identity(base, tpr, ppv):
    return base / (1 - base) * (1 - ppv) / ppv * tpr
def feature_of_column(ref):
    return np.repeat(np.array(d161.NUM + d161.CAT),
    [1] * len(d161.NUM) + [len(np.unique(ref[c])) for c in d161.CAT])
```

- `base / (1 - base) * (1 - ppv) / ppv * tpr` — evaluated left to right: odds, times the false share of the selected over the true share, times TPR.
- `d161.NUM + d161.CAT` — two lists joined: five numeric names, then the categorical ones.
- `[1] * len(d161.NUM)` — list repetition: a 1 for each numeric column.
- `np.repeat(names, counts)` — each name repeated as many times as it has columns: one label per one-hot column, 87 in all.

SYSTEM DIAGRAM



Three criteria and the identity that links them.

Segment 2 — The experiments

demo_measure: a lambda for the bootstrap, and a shuffled-label control

CODE

```
reps = d163.bootstrap_reps(lambda a, b_, c: gap(a, b_, c, mt), (yv, d, sv), B, seed=1)
lo, hi = d163.percentile_ci(reps); obs = gap(yv, d, sv, mt)
rng = np.random.RandomState(2)
ctl = [gap(yv, d, rng.permutation(sv), mt) for _ in range(200)]
```

- `lambda a, b_, c: gap(a, b_, c, mt)` — a small unnamed function of the resampled arrays; mt is taken from the loop.
- `(yv, d, sv)` — a tuple of arrays; Day 163's bootstrap resamples the same rows of all three together.
- `rng.permutation(sv)` — the sex labels shuffled: groups of the same sizes, with no real difference between them.

demo_unaware: dropping columns by feature name

CODE

```
cols = ~np.isin(feat, drop)
w, _ = d161.fit_logistic(Xo[fi][:, cols], y[fi]); p = d161.predict_logistic(w, Xo[va][:, cols])
c2 = cols & (feat != "sex")
w2, _ = d161.fit_logistic(Xo[fi][:, c2], s[fi])
auc_s = d117.roc_auc(s[va], d161.predict_logistic(w2, Xo[va][:, c2]))
```

- `~np.isin(feat, drop)` — True for every column whose feature is not in the drop list; ~ flips the mask.
- `Xo[fi][:, cols]` — the fitting rows, then only the kept columns.
- `cols & (feat != "sex")` — & combines two boolean masks element by element.
- `s[fi]` — sex as a 0/1 target: the same logistic regression now predicts sex instead of income.

demo_thresholds: rules as a dictionary of dictionaries

CODE

```
rules = {"one threshold 0.5": {g: 0.5 for g in GROUPS},
         "parity (equal selection)": {g: threshold_for_rate(p[A & (sv == g)], sel) for g in
             GROUPS},
         "equal opportunity (equal TPR)": {g: threshold_for_tpr(p[A & (sv == g)], yv[A & (sv ==
             g)], tpr) for g in GROUPS}}
for name, thr in rules.items():
    d = decide(p[Bh], sv[Bh], thr); R = group_rates(yv[Bh], d, sv[Bh])
    row = [np.mean(d == yv[Bh])] + [R[g][mt] for g in GROUPS for mt in METRICS] + [thr["Female"],
        thr["Male"]]
    res.setdefault(name, []).append(row)
```

- `{g: threshold_for_rate(...) for g in GROUPS}` — a dictionary comprehension: one threshold per group, chosen on half A only.
- `A & (sv == g)` — the rows in half A and in group g.
- `res.setdefault(name, []).append(row)` — make an empty list the first time a name is seen, then append to it.
- `[R[g][mt] for g in GROUPS for mt in METRICS]` — two for clauses: women's four rates, then men's, in one flat list.

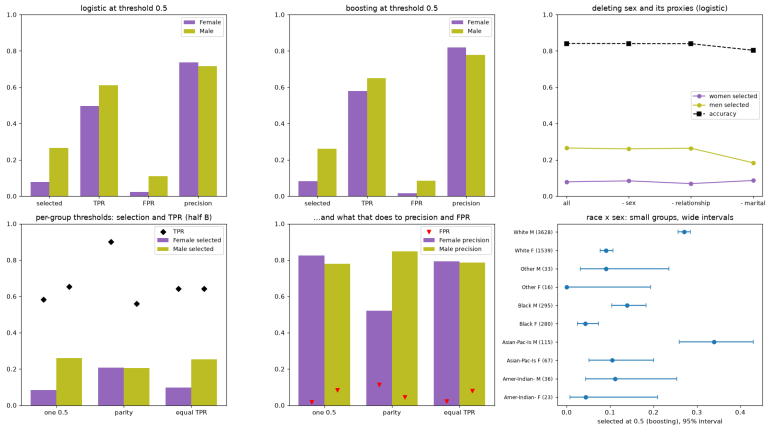
demo_small: Wilson beside the bootstrap

CODE

```
m = (race == r) & (sv == g); n = int(m.sum()); npos = int(yv[m].sum())
lo, hi = d163.wilson_ci(d[m].sum(), n); lo = max(lo, 0.0)
reps = d163.bootstrap_reps(np.mean, (d[m].astype(float),), B, seed=4)
blo, bhi = d163.percentile_ci(reps)
tpr = f"{d[m & (yv == 1)].mean():.3f} (of {npos})" if npos else "undefined (nobody positive)"
```

- `(race == r) & (sv == g)` — both conditions at once: one race and one sex.
- `lo = max(lo, 0.0)` — floors the lower end at 0.
- `(d[m].astype(float),)` — a one-element tuple: the trailing comma makes it a tuple, which the bootstrap expects.
- `... if npos else "undefined (nobody positive)"` — a conditional expression: 0 positives counts as False.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.quantile(x, q)` — the value with a share q of x below it.
- `np.isin(a, values)` — True where a's element is in values.
- `np.repeat(names, counts)` — repeat each item its own number of times.
- `rng.permutation(x)` — a shuffled copy.
- `dict.setdefault(k, [])` — the value for k, created empty if missing.
- `lambda args: expr` — a small unnamed function.

End of Day 166 syntax reference. For the experiments and what they showed, see the Day 166 complete guide.